

Devolución TP2

Grupo 1

Demo

- Falta volver para atrás en anuncios
- Falta eliminar filtros
- Chips en la pagina de restaurante no son clickeables. Si lo son en la búsqueda.
- Hacer reserva no esta en un lugar mas claro. Esta escondido en la solapa de información.
- No hay notificación explícita de los announcements (ni mail al estar suscrito ni aviso in app).
- No hay edición de reseña (las reseñas se pueden agregar extra cada 7 días. Es una decision de negocio rara).
- Falta confirmación de eliminación de review.
- No hay mail de que te borraron una reseña.
- No hay razón de baneo para el dueño en la pagina (solo avisan por mail) pero si te aparece que esta baneado.
- Podría mejorarse el UX de selección de horario de atención.
- Podría haber una pagina dedicada a administración de reservas, como para meseros que tienen que validar su reserva.
- Puedo acceder al /ban de un restaurante a pesar de no ser mod o admin. Tira correctamente 403 al intentar banear con el POST.
<http://pawserver.it.itba.edu.ar/paw-2025a-01/restaurants/1/ban>
- Como máximo se pueden hacer reservas de hasta 10 personas. Este número parece ser arbitrario. Debería poder configurarse por restaurante + tener un indicador mas obvio de cual es el tamaño máximo de reserva.

Código

- Siguen teniendo el schema.sql a nivel de webapp. **Este es un error mencionado en devolución tp1.**
- Varios mal usos de excepciones y manejo de excepciones planos (o IllegalArgumentException) en vez de excepciones custom.
 - Están catcheando NoSuchElementException en el UserDetailsService. Esta RuntimeException solo se utiliza a nivel de estructura de datos para indicar que no existe el elemento (al hacer next de un iterador sin elementos). Se esta catcheando el error incorrecto o en una capa superior lanza un error no indicado.
 - El ExceptionHandler esta manejando DataAccessException y BusinessException pero recibe como exception a nivel de parámetro ImagePersistenceException, loggeando excepciones incorrectas como ImagePersistenceException.
- Configuran el taskExecutor con un queueCapacity de 25, limitando arbitrariamente la cantidad de tareas que pueden ser encolados con @Async.
- Cachean imágenes con ConcurrentMapCacheManager, que sin una estrategia apropiada de eviction causa que se pueda aumentar innecesariamente el uso de memoria de su sistema.
- En toda pagina con paginación no validan que el page o size no sean negativo, generando un error de 500.
- Tienen una función getHomeModel de homeDisplayService que se encarga de popular un Map<String, Object>. Este getHomeModel acopla comportamiento del ModelAndView al service. Esta función es difícil de usar, no indica que tipos de datos devuelve (usa Object como value del map) y no es útil en general. Adicionalmente, la sola existencia de un `HomeDisplayService` es un error conceptual. La idea de un "home display" es un concepto de frontend (webapp) y no corresponde a esta capa. Los servicios debieran definirse en función de casos de uso y del dominio que modelan. Si se decide rediseñar por completo la home page, no debiera ser necesario modificar un servicio. **Esto es un error conceptual grave.**
- Lógica de negocio en el controller. **Estos son errores conceptuales graves.**
 - Hacen varias llamadas a menuService.addItem desde el controller y realizan chequeos a nivel Controller. Pasa devuelta en editMenu, solo que aún mas grave ya que hacen un findById.

- Obtienen Reservation y ReservationTurnInstance antes de lanzar el modifyReservation. Esto es lógica de negocio en el controller.
- editRestaurant realiza multiples updates a nivel de controller. Este error ya fue corregido en una anterior entrega.
- Validan un form a mano en el controller en vez de usar un custom validator. **Este error ya fue corregido previamente en una anterior entrega.**
- Al crear una imagen permiten JPEG, WEBP y PNG. Sin embargo getImage siempre setea como ContentType JPEG. Debería correctamente setear el ContentType correspondiente.
- Buen uso del decorador que valida que hay capacidad adecuada para una reserva.
- Falta de uso de final en UserReservationDTO para campos no modificables, no cumple con buenas practicas de Effective Java.
- Bien uso de custom exceptions y enums (1) para ordenamiento.
- En el servicio de email no encapsulan lo suficiente la lógica de envío de email, encargando a otros servicios de rellenar sujeto, elección del template y que variables utiliza para popular. Esto genera un altísimo acoplamiento entre servicios. El propio servicio de email debería encargarse de pedir que datos necesita para popular el email y que template seleccionar, con una función por cada email a mandar.
 - Esto es mas claro en el createReservation, en el cual tienen sólidamente 10 lineas solo reservados para enviar el mail.
- Automatic unboxing de tipo Long a long sin chequeo de null.
- Funciones publicas del service sin Transactional(readonly=true) en ProfileServiceImpl
- Están usando findById multiples veces para fetchear relaciones 1:1 que pueden tranquilamente ser obtenidos mediante gets del modelo en si. Es redundante.
 - Ejemplo, Reservation tiene turn instance y template y restaurant, al traer las reservations hace un for por todas las reservations, después findById de TurnInstance, findById de template, etc.
- Al cancelar una reserva piden el userId para validar ownership de la reserva a nivel de service. Esta lógica podría ser delegada a Spring Security con un

.access o @PreAuthorize

- Usan varias veces una representación String de fecha en vez de utilizar LocalDateTime o LocalTime. Esto se lo ha mencionado en la anterior entrega en otro caso.
 - Al crear un restaurante usan 2 Map<String, String>, el primero representando por cada día de la semana el horario de apertura y el segundo lo mismo para el horario de clausura. Esto no es una buena representación de un rango de horario de funcionamiento.
- Utilizan Mockito.verify, lo que testea implementación y no comportamiento. **Esto es un error conceptual grave.**
- En varios tests están validando ausencia de error con assertTrue(true). Esto es un mal diseño de un test unitario. Sería mejor hacer que la función devuelva algún valor para verificar su resultado correcto.
- Precaria cobertura de tests en funciones con pesado lógica de negocio (createBatchTemplates, todo RestaurantService). Ningún test en funciones de queries del dao complejas (findByFiltersCount, getRecommendedRestaurants). Mas allá de esto buena cobertura de test en daos.
- Varias funciones en los daos que devuelven X for Restaurants en un Mapa cuando estos pueden ser simplemente creados llamando la respectiva función del modelo.
 - Ejemplo, getFoodTypesForRestaurants, cuando en Restaurant ya existe getFoodTypes(),

Seguimiento

- Hacen correcto uso del plane y bien el uso de chores y bugs.

Nota

3

Promedio: 5

Grupo 2

Demo

- En el formulario de creación de empresa, el error de maxlength de descripción y teléfono no aclara el límite (falta parametrizar)
- El teléfono se puede crear como una secuencia de letras
- Al cambiar la página de gestión de usuarios, aparece un botón para borrar el usuario logueado que no debería estar (es correcto que no haga nada aunque está visible)
- Falta restringir el valor que se le pasa al queryParam page
 - page=-1 arroja un 500
- La descripción no está bien encodeado
- No es consistente el uso del asterisco para los campos obligatorios
- Puedo acceder a tareas de otros usuarios
- La duración de tareas parece estar módulo 24
- La periodicidad aparece en inglés en una página en español
- Le agregaría un filtro por estado crítico a la vista de stock
- El cambio de contraseña podría hacer login
- Algunos links de los mails van a un 404 (la URL usada está mal formada)

Código

- Spring security ya ofrece la funcionalidad de parsear path params para pasarlos a funciones de autorización, haciendo que no necesiten parsearlos ustedes
- Falta una mayor granularidad en el uso de Spring Security. Por ejemplo, cualquier usuario autenticado puede acceder y modificar todas las activities. Sólo los administradores pueden eliminar usuarios pero de cualquier empresa. También los lleva a hacer verificaciones en los controllers
- El uso de ManagementData parecería indicar que fue creado para encapsular todos los modelos requeridos por el endpoint /management. Pero terminan acoplando a nivel de servicio qué se muestra en las vistas.

- Ciertos emails que se reciben como requests param necesitan de verificación por parte del backend de que son emails.
- Role, day y status debieran ser enums
- Tienen validaciones en controllers y en servicios que pertenecen en los forms.
- Hacen chequeos de null por objetos traídos de Optionals.
- En el formulario de SupplyStock, faltan validaciones de tamaño y tipo para el MultipartFile image_content (que debiera ser imageContent).
- En SupplyAndStockController.handleForm, las distintas ejercitaciones a servicios deberían estar encapsuladas en un único método de la creación del Supply.
- No están aprovechando la nueva funcionalidad provista por Hibernate. Por ejemplo, en CompanyInvitationServiceImpl, piden cada campo de una CompanyInvitation y las verifican por separado, cuando podrían traerse la invitación y tener garantías sobre sus campos.
- El manejo de errores es poco claro. En CompanyUserServiceImpl.createCompanyForCurrentUser, la no existencia de un usuario (cuando lo suponemos logueado), hace que la función retorne, sin manera de distinguir este caso de la correcta creación de la empresa. En CurrentCompanyServiceImpl.getCurrentCompanyId, el mismo caso o la no existencia de una empresa retornan 0, por lo que la inexistencia se esconde y se continúa con la ejecución.
- Falta cobertura de testing para TaskScheduleServiceImpl y TaskServiceImpl.
- Hay métodos que no utilizan el modelo 1+1. **Esto es un error grave.**

Seguimiento

- Hacen uso correcto del plane

Nota

6

Promedio: 6.5

Grupo 3

Demo

- Durante la demo se vió un server error
- Los botones no tienen padding, terminan donde termina el texto
- Al apretar el botón de add card, el mismo muestra como un loading, pero los otros botones siguen siendo clickeables. Debiera en todo caso deshabilitar los 3.
- La lupa en el search no está alineada verticalmente
- Los botones para completar el deck no están alineados, no tienen margins, no tienen código de colores
- En general la UI tiene muchos problemas
- El modal de creación de un deck random tiene todo super chico y apretado. Innecesario.
- El desplegable (3 puntitos) de un deck muestra textos desalineados, y hovers que exceden el ancho del popup.

Código

- Es extremadamente llamativo el peso de su repositorio y por tanto lo lento que es operar con el mismo. Los dos principales culpables son:
 - Por un lado incluyen copias de todas las versiones de una font "Fira Sans". Son 20 variantes, a razón de 0.4 / 0.5 MB por variante. Esto es completamente innecesario y hasta ineficiente, cuando dicha fuente está disponible en Google Fonts para usar directo desde su CDN. <https://fonts.google.com/specimen/Fira+Sans>
 - Por otro lado, embeben un fatjar `google-java-format-1.15.0-all-deps.jar`, con un script para su ejecución manual. Esto es innecesario; de mínima ya que el script hace la descarga, podría estar ignorado el directorio `tools`, pero aún mejor, existen plugins de Maven para hacer esto: <https://github.com/google/google-java-format?tab=readme-ov-file#third-party-integrations>
- Hay mucha lógica de negocio en controllers, **esto es un error conceptual grave que ya había sido indicado en la primer entrega.**

- No se hace un buen uso de la base de datos, con métodos que retornan ids, que luego deben ser interpretados y consultados por separado. **Esto ya había sido indicado en la primer entrega.**
 - Para peor, esto denota un modelado del dominio incorrecto, especialmente bajo el uso de JPA, donde los ids pasan a ser un detalle de implementación al modelar relaciones.
- La aplicación trae listas de elementos sin paginación, y por tanto sin garantías de performance.
- Hay tests que no son unitarios, usando más de un método de la class under test. **Esto es un error conceptual grave.**
- Utilizan el paquete `java.sql` en capa de modelos. **Esto ya había sido indicado en la corrección anterior.**
- Siguen omitiendo modificadores de acceso, sin que parezca ser por diseño, ej: `CommunityPostWithUserData`. **Esto ya había sido indicado en la corrección anterior.**
- En múltiples ocasiones, una sola operación (incluso, una sola llamada a un DAO), termina ejecutando N queries a la base de datos. **Esto es un error conceptual grave.**
- Las queries que sí pagan, en general lo hacen sin usar el modelo 1+1. **Esto es un error conceptual grave.**
- Las entidades están mal mapeadas. No sólo que mapean las relaciones mapeando el id pelado (y no referenciando entidades), ni siquiera se establecen las foreign keys correspondientes. Son columnas comunes, sin semántica o constraints. **Esto es un error conceptual grave.**

Seguimiento

- Quedaron varias cards en progreso o en todo.
- Bien uso de las estimaciones de features.

Nota

1

Promedio: 3

Grupo 4

El equipo entrega en forma muy tardía a las 20:52.

Demo

- Al registrarme no soy redirigido a donde estaba
- Tienen imágenes rotas "de sprints pasados" que nunca arreglaron
- Por qué hay un toggle para "administrar mis intenciones de viaje"? ya tienen un modal de confirmación...
- Por qué no puedo editar / cancelar mis solicitudes de viaje?
- No puedo pedir más de 1 slot de una vez? (viaje con mi amigo)
- Intención de viaje no necesita un scheduler... alcanza con el check al momento de crearse un viaje
- Los tabs en mis viajes están amontonados, sin orden...
- Los mails salen apuntando a localhost
- La UX es confusa. El mayor botón visible ("Nuevo viaje") es para conductores, no para usuarios comunes... que si quieren un viaje tienen que ir al search...
- No hay forma fácil de convertirse en conductor... salvo adivinar lo que hace "Nuevo viaje"

Código

- El archivo schema.sql debería estar en el modulo de persistence y no en el modulo de webapp. **Esto ya ha sido marcado en la primer entrega.**
- Es innecesario tener que almacenar y utilizar la variable `webapp_base_url` en la capa de webapp para los JSP, `<c:url>` ya resuelve esto. **Esto ya ha sido marcado en entregas anteriores.**
- Imprimen mensajes directamente sin utilizar `<c:out>` sin escapar el contenido, lo que los hace vulnerables a XSS. **Esto es un error grave que ya ha sido marcado en entregas anteriores.**
- Poseen lógica de negocio en los controllers. **Esto es un error conceptual grave que ya ha sido indicado en entregas anteriores.**

- Hacer try catch de exception en los controllers no es correcto, estos errores deberían ser manejados con el Exception Handler y retornar el error correspondiente. Si desean hacer alguna validación de negocio se deben utilizar custom validators. **Esto ya ha sido marcado en la entrega anterior**
- Solo poseen validación de roles en las rutas, falta un nivel mas fino de control de accesos para evitar que un usuario pueda realizar acciones sobre recursos que no le pertenecen. Por ejemplo, es posible aceptar un trip request para cualquier viaje siendo un conductor. **Esto es un error grave que ya había sido indicado en correcciones anteriores.**
- En el `HomeServiceImpl` el método `parseMaxPrice` debería ser `private`. Hay mas lugares donde existen método públicos que deberían ser privados. **Esto ya había sido indicado en correcciones anteriores.**
- La sola existencia de un `HomeService` es errónea. El concepto de "home page" concierne al frontend (webapp), los servicios debieran definirse en concepto de casos de uso y el dominio que representa el sistema.
- Hacen uso de boxed primitives sin razón, en escenarios donde no sólo el valor `null` no parece tener sentido, sino que el código ni siquiera lo contempla, haciendo auto-unboxing de estos valores en forma generalizada y siendo susceptibles por tanto a `NullPointerException` s. **Esto ya había sido indicado en la corrección anterior.**
- Utilizan `LocaleContextHolder` (locale del usuario activo) para el envío de mails en lugar de utilizar el locale del destinatario. **Esto ya había sido indicado en la entrega anterior.**
- El uso de `SecurityContextHolder` y la subsiguiente lógica para obtener el usuario es repetitivo, y se podría evitar usando un `@ControllerAdvice`.
- No hacen uso correcto de `@Transactional`. **Esto ya ha sido indicado en la corrección anterior.**
- No siguen la convención de nombres de Java, con algunos paquetes incluyendo nombres con mayúscula.
- Hacen uso sin razón de `em.unwrap(Session.class)` y `addScalar`

Seguimiento

- Quedaron tareas In Progress o en Todo.

Nota

1

Promedio: 3.5

Grupo 5

Demo

- Los usuarios sin reseñas se muestran como teniendo 0 estrellas
- Tras registrarte te manda siempre a la home y no a retomar el flujo que iniciaste
- La pantalla de "Verificación requerida" permite reenviar el código, pero no ingresarlo.
- El mail no tiene la misma paleta de colores que el sitio, parece ser de otra aplicación.
- Las imágenes en los botones de Cambiar contraseña y editar perfil se pisan con el texto, faltan paddings.

Código

- Corrigen los problemas detectados en la primer entrega en forma satisfactoria.
- Hacen queries paginadas sin usar el modelo 1+1, por lo que potencialmente se están trayendo la tabla entera para paginar en memoria. **Esto es un error grave.**
- Hay tests de la capa de persistencia que no validan el estado de la base de datos.
- El repositorio posee archivos del IDE (*.iml)

Seguimiento

- Hacen correcto uso del plane y estimación de features.

Nota

8

Promedio: 6.5

Grupo 6

Demo

- Bien por tener multiples filtros
- Bien que en la url se ven los parámetros de los filtros
- Cuando creo un perfil el instagram es obligatorio pero en el frontend no tiene el *
- Bien que el delete de un evento tiene confirmación
- Esta raro el edit del announcement y solo me deja editar el announcement mas antiguo
- Comentar un evento que estoy rechazado tira 500 (que es custom)
- El share me copia la url con todos los query params, quedando una url extraña <http://pawserver.it.itba.edu.ar/paw-2025a-06/event/77?commentPublished=true>, solo debería dejar el query param de paginación
- El editar de la review esta raro como el del announcement, seria mejor ocultar la reseña que esta siendo editada de la lista.
- El estado pendiente de eventos pasados no hace mucho sentido, tiene más sentido que sea "vencido"

Código

- El repositorio posee archivos generados (.mvn)
- Concatenan textos en la vista en lugar de interpolar strings. **Esto ya se marcó en la primer entrega.**
- Realizan chequeos de security en el controller
- La lógica para mostrar un botón u otro si el usuario es dueño del recurso es parte de spring security y podría ser resuelto con taglib de spring-security
- Hay métodos que no utilizan el modelo 1+1 para colecciones. Si bien las mismas no poseen relaciones *ToMany es mejor utilizar el método 1+1 ya que, si el modelo en cuestion cambia, es menos propenso a errores.

- Utilizan boxed primitives y no parece ser por diseño. **Esto ya se marcó en la primer entrega.**

Seguimiento

- Hacen un correcto uso de Plane.

Nota

8

Promedio: 7.5

Grupo 7

Demo

- Podría estar marcado cual equipo es cada color para ser mas claro.
- Estaria bueno mostrar información agregada en las tabs, por ejemplo cantidad total de amigos.
- Muestran la opción de cambiar el lenguaje cuando se visita el perfil de un usuario. Solo es un error de UI ya que no se puede realizar ningun cambio (lo cual es correcto).
- Al ingresar a algunos events se produce un error 500 (por ejemplo event/22).
- Algunos textos internacionalizados muestran la clave, por ejemplo "invite.friends.send".
- Estaría bueno poder ver las invitaciones a grupos tanto del lado del equipo como quien las recibe. Ahora este flujo se completa por email.

Código

- Tienen TODOs en el código como por ejemplo

```
/// TODO ya mismo tengo que entender que hace esto. lo hizo gpt  
/// y si no entiendo lo que hace no lo  
/// pienso usar
```

Tienen secciones de código bastante desprolijas con comentarios o TODOs.

- Cumplen a medias con la no instanciabilidad de clases utils. Revisar ítem 4 de Effective Java. **Esto ya se marcó en la primer entrega.**
- Realizan commits con mensajes poco claros, como por ejemplo: "aaaaaa".
- Utilizan clases de `java.sql`. La existencia de una base de datos es un detalle de implementación del DAO. **Esto ya se marcó en la primer entrega.**
- Utilizan boxed primitives y no parece ser por diseño. **Esto ya se marcó en la primer entrega.**
- Definen clases que luego no utilizan.
- Definen un enum para Location pero luego lo definen como String en el modelo.
- El método `getNumberOfMatchesPlayedLastMonth()` de la clase `Session` realiza el filtrado en Java en lugar de hacerlo directamente sobre la base de datos.

Seguimiento

- Hacen un correcto uso de Plane.

Nota

7

Promedio: 7

Grupo 8

Demo

- No se puede recuperar la contraseña
- Bien por dar feedback al navegar la aplicación
- Tienen algunos textos sin i18n
- Están asumiendo que los archivos de pago son siempre pdfs y no funcionan con otros tipos de archivos

- En el panel de administracion estaría bueno ver la puntuacion del usuario asi como las reseñas

Código

- Las vistas concatenan strings en lugar de interpolarlos. **Esto es un error grave. Esto ya se marcó en la primer entrega.**
- Realizan validaciones en los controllers en lugar de hacerlo a través de Forms.
- Las clases piden que les inyecten atributos que luego no usan. **Esto ya se marcó en la primer entrega.**
- Los tests no son unitarios ya que usan la misma class under test para el setup. **Esto es un error conceptual grave.**
- En los tests de persistencia no validan el estado final de la base de datos.
- Expresan la intención de no hacer N+1 queries, pero inmediatamente luego hacen N queries.
- Utilizan clases de `java.sql`. La existencia de una base de datos es un detalle de implementación del DAO.
- Overridean hashCode para que siempre retorne el mismo valor.

Seguimiento

- Hacen un uso correcto de Plane.

Nota

6

Promedio: 5

Grupo 9

Demo

- Usan dos términos (softcover y paperback) para el mismo concepto
- Edición de posteo: cambio de imagen arroja un 500 y no cambia la misma

- Buena experiencia el multiselect con autocomplete de autores
 - Se podría agregar en el filtro para la selección de ubicación
- Intento de XSS en reviews termina en un tira un 400 pero se genera la review
- El enter en location no me envia el form de búsqueda
- Unsuspend una cuenta me lleva a una página de 500 pero sí realiza la acción y envía el mail
- Al contraofertar, el mensaje de error por no elegir libros a recibir no está internacionalizado

Código

- La query de /books/authors no está paginada, trayéndose una lista de tamaño desconocido y que depende del threshold de similaridad y de cuántos autores se hayan cargado.
- /books/authors sin el QueryParam query muestra una página 500. Revisar el manejo de la excepción MissingServletRequestParameterException.
- La lógica de cuándo una publicación se puede o no pausar (usuario registrado es dueño, no está pausado y no tiene intercambios pendientes) debería estar delegada a la capa de servicios.
- Aceptan imágenes jpeg, png y webp pero al devolverlas siempre definen el contentType como jpeg.
- Hay instancias donde, pudiendo arrojar sus excepciones custom, arrojan un NoSuchElementException genérico al usar orElseThrow().
- Salvo por ReportDaoJpaImpl, buena cobertura de tests en general.
- Las clases PageableQueryBuilder y PagingJpa son muy buenos recursos. El único caveat es que no tener implementado el método getId pasa a ser un error de Runtime en vez de compilación.
- El uso de Dtos para mandar datos entre los servicios y el servicio de mailing parece innecesario, siendo que estos pueden utilizar los modelos.
- Tiene puntaje a favor de la primera entrega

Seguimiento

- Hacen uso correcto de Plane

Nota

9

Promedio: 8.5

Grupo 10

Demo

- Bien por tener multiples filtros
- Bien que en la url se ven los parámetros de los filtros
- El attend de un event esta un poco escondido en el flujo
- Muy buena la vista de reportes, los accionables que te ofrece son los esperados
- La funcionalidad del admin es muy completa

Código

- Poseen lógica de negocio en los controllers. **Esto es un error conceptual grave.**
- Utilizan `pageContext.request.isUserInRole('ADMIN')` en lugar de utilizar la taglib de spring security
- Utilizan clases de `java.sql`. La existencia de una base de datos es un detalle de implementación del DAO.
- Hay mucho uso de EAGER en el mapeo de relaciones ManyToOne, algunas de ellas innecesarias debido a que no se utilizan siempre dichos datos.
- Si bien hay una cantidad adecuada de tests en servicios, falta coverage en algunos casos como createJourney que es una funcionalidad core de la app

Seguimiento

- Buen uso de la herramienta

- Agregar paginación a una vista que le faltaba paginación no es una chore porque le agrega valor al usuario

Nota

8

Promedio: 7.5

Grupo 11

Demo

- Muy bien de features
- Muy bien los componentes custom implementados
- Bien los flujos nuevos, el perfil de médico y el sistema de oficinas
- No presenta errores o dead ends

Código

- Se mezcla el uso de `optional.empty` o `id = -1` para la detección de una imagen no existente. **Esto ya había sido marcado en la primer entrega.**
- Hay mapeos de colecciones `toMany` en donde la relación es eager (ej: `appointmentFiles`) y es innecesario cargar todos los archivos cada vez que se levanta un `Appointment`.
 - Para empeorar este caso, los archivos deberían accederse vía un endpoint aparte y no ser arrastrados enteros vía la relación (cosa que impide, por ejemplo, el caching)
- Los controllers contienen lógica de negocio, (ej: `doctorAppointmentDetails`). **Esto es un error conceptual grave**
- Excelente cobertura de tests
- Bien el mapeo de hibernate

Seguimiento

- Ok de seguimiento

- Ok de plane

Nota

7

Promedio: 7.5

Grupo 12

Demo

- Cambiar tamaños no anda
- Bien de features
- El flujo principal está correcto

Código

- Buen uso del builder pattern
- Hay relaciones mapeadas con JPA que pueden crecer indefinidamente. EJ: requests de AllocationPost
 - Esto es agravado ya que no se utiliza FetchType.Lazy
 - Esto es agravado ya que se llama a esta colección para hacer solamente un `.size()` lo cual consume memoria innecesariamente.
- Buen uso de `@Formula` para evitar mapeos innecesarios para queries muy usadas.
- La capa de servicios no tiene buena cobertura de tests, servicios importantes como SizeService no tienen tests.
- Buena cobertura en la capa de persistencia de tests, faltan tests para casos de error solo se teastean casos felices.

Seguimiento

- Ok de seguimiento
- Ok de plane

Nota

9

Promedio: 7.5

Grupo 13

Demo

- Aunque la home por defecto no tiene filtros (correcto), al pasar de página aplica automáticamente un mínimo de \$2 y un máximo de \$1000000.
- Al buscar por nombre hace lo mismo.
- Intentar acceder a "Reservas de mis equipos" obtengo un 403. Nunca un link presentado al usuario debiera llevarlo a un error de este tipo.
- El que puedas unirte unilateralmente a cualquier equipo y ver sus reservas y miembros es peligroso. Debiera tener mejor flujo de gestión de permisos.

Código

- Clases como `EmailTemplates` no debieran ser instanciables.
- Omiten modificadores de acceso y no parece ser por diseño.
- Paginan sin usar el modelo 1+1. **Esto es un error conceptual grave.**
- El repositorio incluye archivos de backup de vim (`*.~`)

Seguimiento

- Buen uso del plane.

Nota

9

Promedio: 7

Grupo 14

Demo

- Mandan 2 mails de onboarding, uno saludandote y uno por creacion de cuenta. Estos podrían estar combinados o tener CTAs mas relevantes en el mail de creacion de enterprise.
- Enums de area no traducidas en la pagina
- Tienen scroll raro tanto horizontal y vertical en anuncios cuando hay pocos anuncios.
- Tienen un efecto on Hover a pesar de no ser clickeable.
- En user/profile si el bio esta vacio se ve mal interpolado. En la pagina de onboarding el bio no es required pero después al editar si.

FEDE

Título: Dueño - Área Asignada: FRONT_END

Bio: "" {0} "" ← como se ve

- Indicar cuantas vacaciones puedo llegar a pedirme. Actualmente no se resetean las vacaciones. En teoría deberían resetearse tras año.
- Te permite crear edificios de longitud mayor a pesar de no permitirte buscar por edificios de mayor longitud. Muestra un error al buscar por edificios ya existentes.
- Falta ir para atrás en un equipo.
- Faltan confirmaciones en cancelaciones o borrados.
- Muchos lugares el enum de area no esta traducido (perfil de usuario, algunos selects)
- Se ven medio extraños los errores al buscar un lugar para reservar. Mueven el layout de una forma fea.
- Las opciones de edificio se muestran de forma textual. "`| <script> alert(22) </script><script> alert(22)</script> | | EDIFICIO |`". No es un buen UX.
- Quedo un mensaje en Sujeto de email "You have left an event" y deberia estar en español.
- Hay algo raro con los timezones, ya que te tira error si intentas buscar espacios libres unas horas despues de ahora.
- Indica "No hay Usuarios que correspondan con tu búsqueda" a pesar de no tener una búsqueda hecha.

- En el login/register no hay forma de volver a la pagina principal.
- Bien cambio de idioma en la pagina.
- Bien las estadísticas de reservas
- Estando con la sesion iniciada, ir a / hace que te tire un 403. Debido a que cada cuenta tiene adjudicado 1 enterprise, podria tranquilamente redirigirte a la pagina correcta.

Código

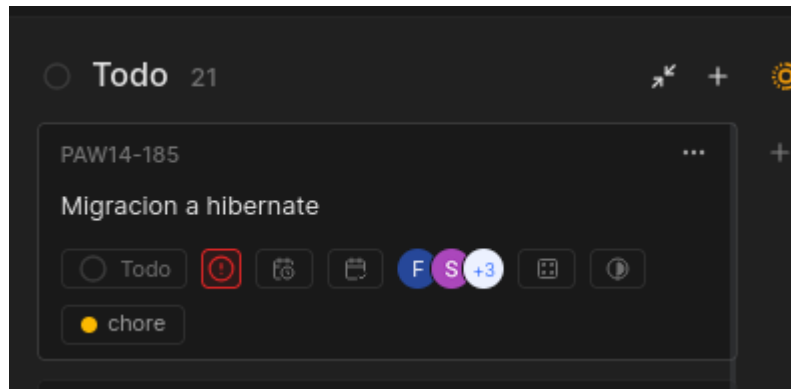
- En el resolveLocale try catchean un NullPointerException en vez de chequear por null proactivamente. Es una mala practica en Java.
- Faltan chequeos de tipos de imágenes y tamaño de imágenes en los forms.
- Varias funciones del controller tienen lógica de negocio. **Esto es un error conceptual.**
- Mal estilo de código: Tienen varios prints en el código, uso de snake case en accept_request
- Tienen una estructura jerárquica de roles que puede ser mejor implementada mediante un AccessDecisionVoter, en vez de agregar authorities escalonadamente. <https://docs.spring.io/spring-security/reference/servlet/authorization/architecture.html#authz-hierarchical-roles>
- Contienen XSS debido a la falta de varios c:url. **Esto es un error grave que ya se les ha corregido en una anterior entrega.**
- Falta de validación de la existencia del valor enterpriseld en varios forms, que después acceden mediante un findById(id).get() sin chequear por su presencia en el Optional. No parecen conocer el correcto uso de Optional. Varios usos de Optional.get() sin chequear por su presencia.
- Crean una funcion de servicio (a nivel de interfaz también) que solo devuelve una lista vacía. No parece ser muy util. Hay otra funcion (getPreviousParams) que te limpia los query params de page y size de una URL. Estos podrían volverse utilidades a nivel de controller.
- Interfaces del DAO en create devuelven Optional, pero en la implementación no hay caso en el cual el Optional es empty. Mal planteo del contrato del create.

- Funciones join y leave de event devuelve ints que valen 0 o 1 para indicar si se unió o no. Estos deberían ser booleans.
- Varios servicios tienen varios gets sin @Transactional(readonly=true)
- Varias funciones de service reciben Long sin manejar la posibilidad de null (Long enterpriseld). También tienen una función que devuelve Boolean en mayúscula pero el DAO devuelve boolean, lo cual genera unnecessary boxing.
- No se validan overlap de peticiones de vacaciones para un usuario. Esto es un error de negocio.
- Hay pocos tests en la capa de servicio en funciones que no son triviales. En particular, AnnouncementServiceImpl, EventServiceImpl. **Esto fue corregido en la anterior entrega.**
- Clase Defaults tiene constructor default a pesar de ser una clase de utilidades. Falta un constructor privado.
- Faltan logs en los ControllerAdvice (para manejo de errores) y a nivel de service loggeando acciones no triviales.
 - También en AmenityHibernateDao están obteniendo logger de una clase foránea. Parece que hicieron copy paste.
- Hay algunas queries con paginación que no utilizan 1+1. A pesar de no tener relaciones eager 1:n, es recomendable seguir utilizando el esquema.
- Hay varias instancias en el cual las queries se siguen realizando de forma nativa, a pesar de poder utilizar HQL para aprovechar más las capacidades de hibernate.
 - Ejemplo: para obtener las estadísticas de las evaluaciones, hacen una native query y después hacen un mapper casteando el objeto a Number.
- Decente cobertura de tests de persistencia, aunque hay nulos tests en el DAO de estadísticas de evaluaciones.
- Buen uso de Javadocs a nivel de dao.

Seguimiento

- Bien el uso de plane, buen uso de features canceladas despues de feedback en ciertos sprints. Quedaron algunos TODOs sin hacer, uno que

parece bastante importante.



Nota

5

Promedio: 6.5

Grupo 15

Demo

- Cuando busco una pagina que va mas allá del máximo me dice "Pagina 55 / 3" seria correcto que marque "Pagina 3/3"
- Estaría bueno un helper text en el numero de teléfono para el create ya que el mensaje de error no me dice mucho
- Falta el mensaje de error en peso altura y fecha de nacimiento
- Falta CTA en la vista de turnos cuando no tengo turnos
- El modal de autorizar se rompe si lo abro una 2da vez
- El mail de estudio recibido me lleva a una pagina con 404
- Los estudios y mis medicos deberían estar paginados

Código

- Los archivos de textos para locales específicos como `en_US` heredan del archivo `en`, no es necesario copiar y pegar todas las keys si no tienen diferencias. **Esto había sido marcado en la primer entrega.**

- Imprimen valores a JSP usando `${...}` y no `<c:out>`, lo que los expone a vulnerabilidades XSS. **Esto es un error grave y había sido marcado en la primer entrega**
- Usan alt texts para las imágenes (lo que es positivo para accesibilidad!), pero los mismos no están internacionalizados. **Esto había sido marcado en la primer entrega**
- Sentencias como `if (user instanceof Patient)` y `else if (user instanceof Doctor)` no son recomendadas, es mejor tener un método abstracto en la clase user y que las subclases Doctor y Paciente lo implementen a su manera; o en su defecto, una implementación default + overrides.
- La sentencia en los controllers

```
if (user == null) {
    throw new UnauthorizedException("User not found");
}
```

corresponde a una validación de seguridad y por lo tanto debería ser manejada por spring security

- No hace falta hacer catch en los controllers, sino utilizar el Exception Controller advice para que resuelva las excepciones
- Poseen lógica de negocio en los controllers. **Esto es un error conceptual grave**
- Métodos como este traen de la base de datos todos los singleShifts de un doctor y realizan un filtrado en memoria. Lo mismo sucede con `getAvailableDays` y `getAuthorizedDoctors` donde se traen toda la colección de doctores autorizados de un paciente.

```
public List<DoctorSingleShift> getActiveSingleShifts() {
    return singleShifts.stream()
        .filter(DoctorSingleShift::getIsActive)
        .sorted((shift1, shift2) -> shift1.getWeekday().compareTo(shift2.getW
        .toList());
}
```

- Lo único paginado en la webapp es la lista de doctores de la home. Hay multiples casos que deberían ser paginados que no están contemplados:
 - Los Appointments pasados

- Los estudios de un paciente
- Los doctores autorizados (este es debatible)
- Los turnos futuros (también debatible)
- Si bien Doctor tiene Insurances como relación OneToMany y FetchType Lazy hace que la paginación funcione sin utilizar el método 1+1, es mas propenso a errores dado que si se modifica a Eager, esto va a causar que la paginación de doctores se rompa. Es recomendable y correcto utilizar 1+1 para evitar este tipo de problemas.

Seguimiento

- Buen uso de la herramienta

Nota

5

Promedio: 6

Grupo 16

Demo

- El proceso nuevo de onboarding está bueno
- El chequeo del máximo de links está solo en front
- Animaciones y responses a feedback del usuario están bien implementadas
- El flujo principal del app está completo.
 - Bien de features

Código

- Omiten modificadores de acceso. **Esto ya había sido marcado en correcciones anteriores.**
- El mapeo de las relaciones está correctamente implementado en JPA.
- Hay mucho uso de EAGER en el mapeo de relaciones ManyToOne, algunas de ellas innecesarias, ej: El episodeReview carga todo el episode, pero

muchas veces eso es innecesario.

- Quedan funciones en la capa de servicios sin cobertura que tienen bastante lógica.
- La cobertura de tests de persistencia es parcial y discrecional. Hay flujos bien cubiertos, hay servicios enteros sin 1 solo test, ej: **FriendJpaDao**.

Seguimiento

- Ok de seguimiento
- Ok de plane

Nota

8

Promedio: 7

Grupo 17

Demo

- Bien por logear al usuario al recuperar la contraseña.
- Al agregar a favorito, aparece un popup del navegador "Could not toggle favorite. Please try again."
- Los likes no funcionan, retornan 404 siempre. El path al recurso es incorrecto.
- En el team ranking, no es del todo claro cuando el equipo es de 1 o mas personas.
- Estaría bueno mostrar las imágenes de perfil de los usuarios en los diferentes lugares de la aplicación, por ejemplo solicitudes de amistad, participantes de torneo.
- En la vista del match, deberían mostrar cuantas partidas debería ganar un equipo-jugador para pasar de ronda.
- Las sesiones de los torneos deberían tener un nombre, para que al mostrar la notificación de la sesión, sea claro para el usuario.

- Debieran notificar los torneos ganados, de la misma manera que lo hacen con las sesiones.
- Además de por fecha, podrían filtrar si el torneo ya está terminado. En la demo al jugar el torneo en el momento, se sigue mostrando como un evento futuro cuando ya fue terminado.
- Al añadir un amigo me tira 500.
- Al reportar un jugador, tira un 404.
- La lógica de reportes no es del todo clara, el admin puede ver un log de reportes, pero no tiene información de cantidad de reportes o motivo.
- Sería adecuado poder filtrar por tipo de notificaciones.
- Al crear una sesión la aplicación se queda cargando infinitamente cuando se hace clic en notificar amigos.
- La validación de resultados de las sesiones no tiene mucha utilidad como está actualmente, el hecho de tener más o menos validaciones no influye en nada en la aplicación.
- Bien por dar feedback al navegar la aplicación.

Código

- Realizan validaciones en los controllers en lugar de hacerlo vía forms.
- Tienen tests vacíos.
- Los tests no son unitarios, ya que usan la misma class under test para setup o verificación. **Esto es un error conceptual grave.**
- Hay métodos de persistencia que cuando crean una entidad devuelven un id en lugar de devolver la entidad creada.
- No realizan paginación utilizando el modelo 1+1. **Esto es un error conceptual grave.**
- Acceden a listas sin una paginación adecuada, pudiendo traer un número violento de registros. Por ejemplo, `getAllFriends`.
- Las credenciales del README no funcionan. Además no se proveen credenciales para el usuario admin.

Seguimiento

- Hacen un buen uso de Plane

Nota

7

Promedio: 7.5